École Supérieure en Informatique

ESI - Sidi Bel Abbès

Lab Report 03

Access Control & ACLs on Linux

Computer Systems and Network Security

Academic Year 2024–2025

Student:
Manel Mostefaoui

Instructor:
Mohammed Salih Bendella

October 22, 2025

Before You Start

Objective: Create users and groups, and assign them to the appropriate groups before starting the exercise.

Step 1: Create users and groups
Run the following commands as `root` or using `sudo`:

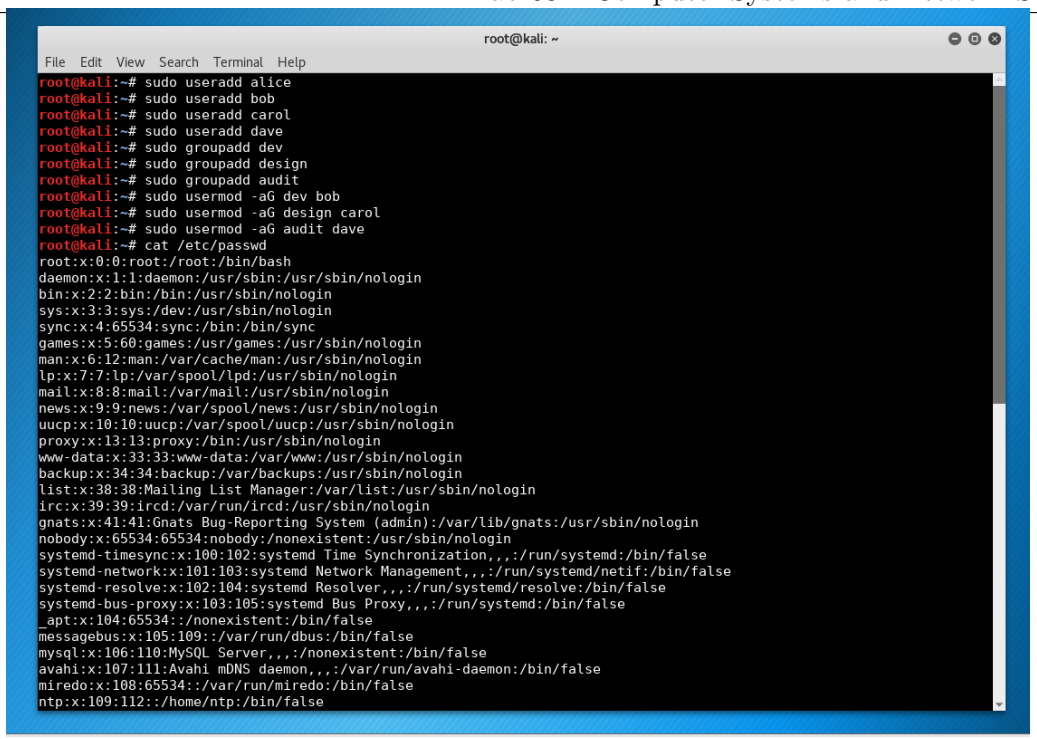
```
sudo useradd alice
sudo useradd bob
sudo useradd carol
sudo useradd dave

sudo groupadd dev
sudo groupadd design
sudo groupadd audit

sudo usermod -aG dev bob
sudo usermod -aG design carol
sudo usermod -aG audit dave
```

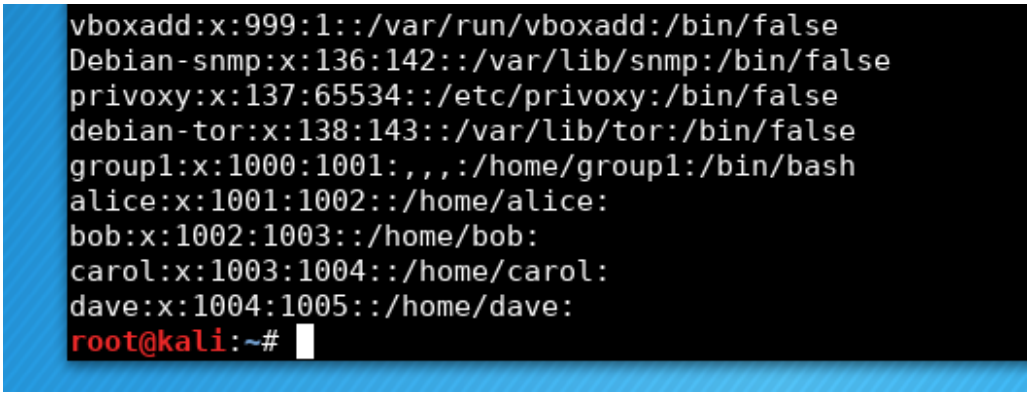
These commands create four users (`alice`, `bob`, `carol`, `dave`) and three groups (`dev`, `design`, `audit`). Each user is then added to their respective group.

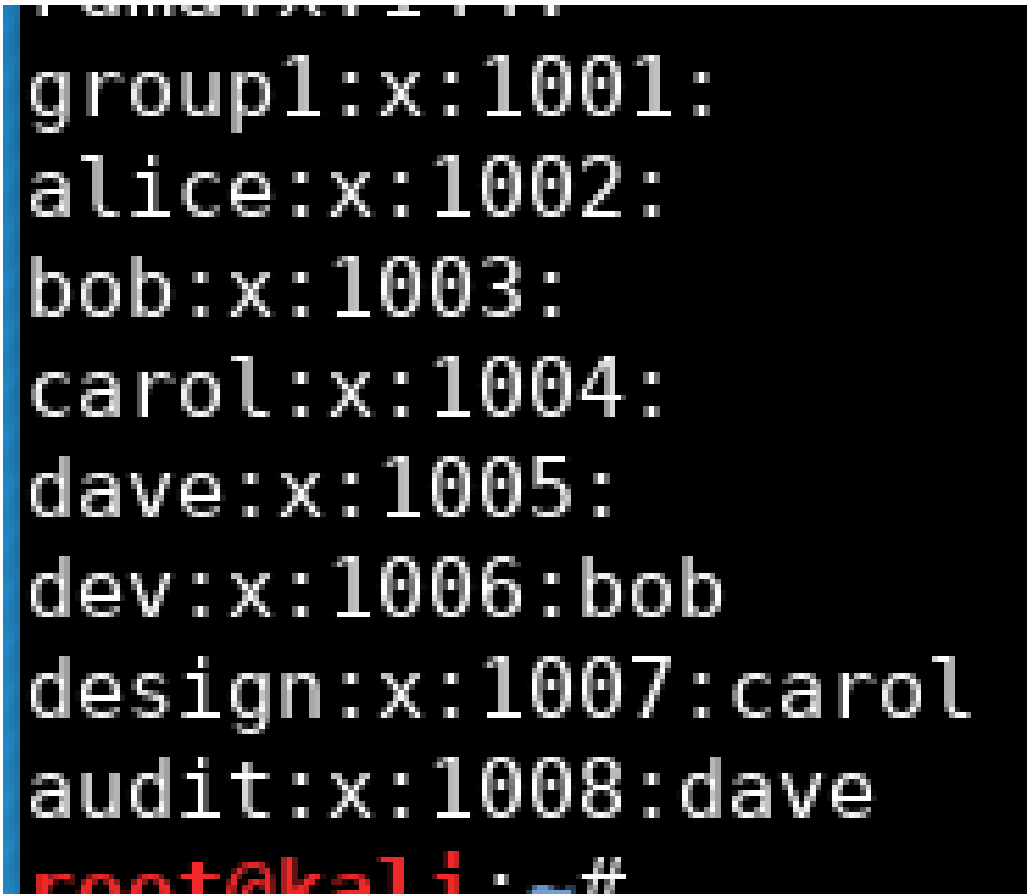
Step 2: Verify created users and groups

```
cat /etc/passwd
cat /etc/group
```

Screenshot 1: List of created users and groups.


root@kali: ~
File Edit View Search Terminal Help
root@kali:~# sudo useradd alice
root@kali:~# sudo useradd bob
root@kali:~# sudo useradd carol
root@kali:~# sudo useradd dave
root@kali:~# sudo groupadd dev
root@kali:~# sudo groupadd design
root@kali:~# sudo groupadd audit
root@kali:~# sudo usermod -aG dev bob
root@kali:~# sudo usermod -aG design carol
root@kali:~# sudo usermod -aG audit dave
root@kali:~# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-timesync:x:100:102:systemd Time Synchronization,,,:/run/systemd:/bin/false
systemd-network:x:101:103:systemd Network Management,,,:/run/systemd/netif:/bin/false
systemd-resolve:x:102:104:systemd Resolver,,,:/run/systemd/resolve:/bin/false
systemd-bus-proxy:x:103:105:systemd Bus Proxy,,,:/run/systemd:/bin/false
_apt:x:104:65534::/nonexistent:/bin/false
messagebus:x:105:109::/var/run/dbus:/bin/false
mysql:x:106:110:MySQL Server,,,:/nonexistent:/bin/false
avahi:x:107:111:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/bin/false
miredo:x:108:65534::/var/run/miredo:/bin/false
ntp:x:109:112::/home/ntp:/bin/false


vboxadd:x:999:1::/var/run/vboxadd:/bin/false
Debian-snmp:x:136:142::/var/lib/snmp:/bin/false
privoxy:x:137:65534::/etc/privoxy:/bin/false
debian-tor:x:138:143::/var/lib/tor:/bin/false
group1:x:1000:1001:,,,:/home/group1:/bin/bash
alice:x:1001:1002::/home/alice:
bob:x:1002:1003::/home/bob:
carol:x:1003:1004::/home/carol:
dave:x:1004:1005::/home/dave:
root@kali:~#


group1:x:1001:
alice:x:1002:
bob:x:1003:
carol:x:1004:
dave:x:1005:
dev:x:1006:bob
design:x:1007:carol
audit:x:1008:dave
root@kali:~#

Exercise 1 – Basic Permission Manipulation

Objective: Manipulate file permissions using basic Linux commands.

Step 1: Create the file

```
touch ~/report.txt
```

This command creates an empty file named `report.txt` in the user's home directory.

Step 2: Show its permissions

```
ls -l ~/report.txt
```

Example output:

```
-rw-r--r-- 1 user user 0 Oct 22 11:45 /home/user/report.txt
```

The owner has read and write permissions, while the group and others have only read access.

Step 3: Change the permissions
We want to give:

- Full rights to the owner (`rwx`)
- Read-only rights to the group (`r--`)
- No rights for others (`---`)

```
chmod 740 ~/report.txt
```

Step 4: Verify the new permissions

```
ls -l ~/report.txt
```

Example output:

```
-rwxr----- 1 user user 0 Oct 22 11:45 /home/user/report.txt
```

Now, the file gives full access to the owner, read-only access to the group, and no access to others.

Screenshot 2: Result of each command.

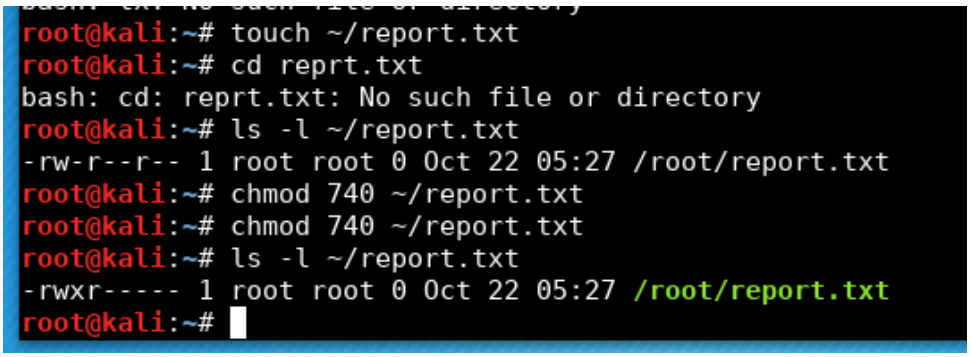


Figure 2: Result of each command for Exercise 1.

Questions:

- **Numeric value corresponding to these rights:** `740`
- **Explanation:**
  - Owner = `rwx` = 7 (4 + 2 + 1)
  - Group = `r--` = 4 (4 + 0 + 0)
  - Others = `---` = 0 (0 + 0 + 0)
- **Is the principle of least privilege respected?**
  Yes. Only the owner has full control, the group can read the file, and others have no access. This ensures no user has more permissions than necessary.

Exercise 2 – Directory Permissions

Objective: Create a directory and assign specific permissions to it.

Commands executed:

```
# Create directory
mkdir ~/tp_acl

# Set permissions: owner=rwx, group=rx, others=none
chmod 750 tp_acl

# View directory details
ls -ld tp_acl/
```

Result:

- Directory `tp_acl` created successfully.
- Permissions set to `rwxr-x---` (numeric value: 750).
- Owner has full control, group members can read and access, others have no access.

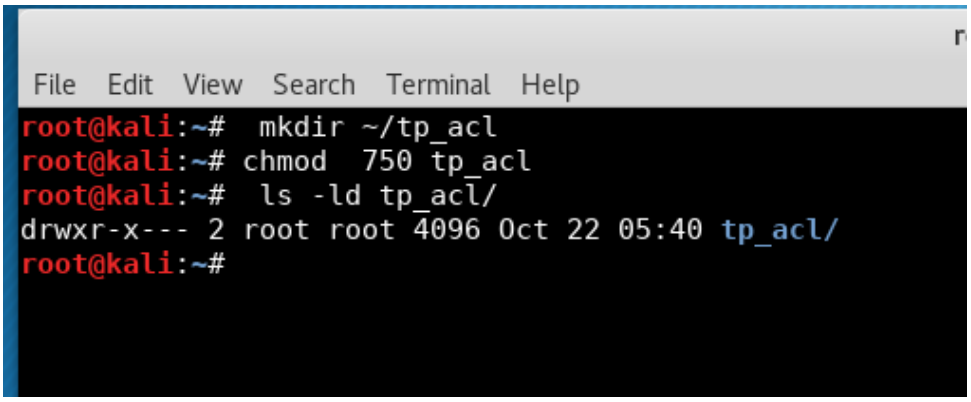


Figure 3: Screenshot 3 – Output of `ls -ld tp_acl/` showing correct permissions.

Question

What is the difference between the execute right on a file and on a directory?

On a file: The execute bit allows running the file as a program or script.
On a directory: The execute bit allows entering (cd) and accessing its contents; without it, even if you can list the directory, you can't open the files inside.

Exercise 3 – Change Owner and Group

1. Change the owner of the file `report.txt` to alice

```
sudo chown alice report.txt
```

2. Change the group of the file to dev

```
sudo chgrp dev report.txt
```

3. Or do both at once

```
sudo chown alice:dev report.txt
```

4. Verify the result

```
ls -l report.txt
```

Screenshot 4: Output of `ls -l report.txt`

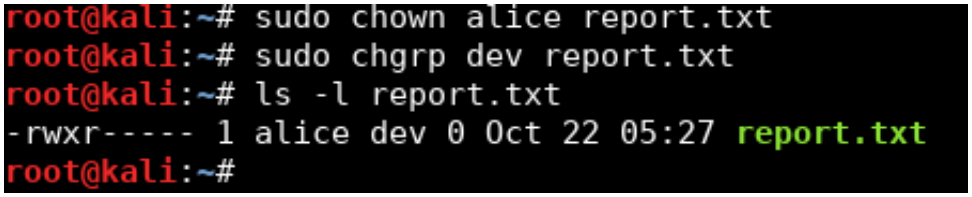


Figure 4: Verification after changing owner and group of `report.txt`

Question: What is the difference between `chown` and `chgrp`?
Answer: `chown` changes the **owner** of a file, while `chgrp` changes the **group** associated with that file.

Exercise 4 – ACL: Fine-Grained Access Control

1. Ensure the ACL package is installed

```
sudo apt install acl -y
```

2. Apply ACLs on the directory `tp_acl`

```
setfacl -m u:bob:rw tp_acl/
setfacl -m u:carol:r tp_acl/
setfacl -m u:dave:r tp_acl/
```

3. Display the ACLs

```
getfacl tp_acl/
```

Screenshot 5: Output of displayed ACLs

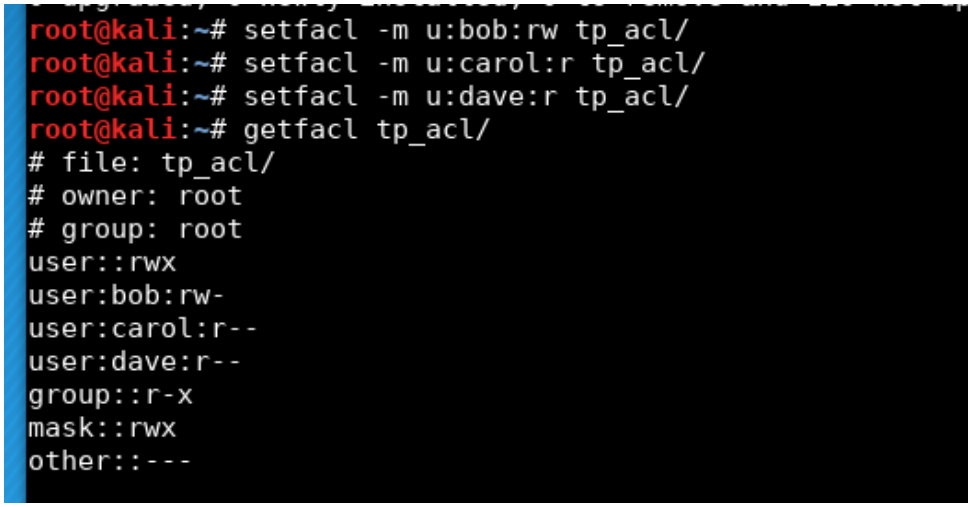


Figure 5: Displayed ACLs for directory `tp_acl`

Question: What is the difference between an ACL and classic Unix permissions?
Answer: Classic Unix permissions define access for only three entities: the owner, the group, and others. ACLs (Access Control Lists) allow defining permissions for multiple specific users or groups, providing more fine-grained control.

Exercise 5 – ACL Inheritance (Default ACLs)

1. Add default ACLs to the directory `tp_acl`

```
setfacl -d -m u:bob:rw tp_acl/
setfacl -d -m u:carol:r tp_acl/
setfacl -d -m u:dave:r tp_acl/
```

2. Create a file inside the directory and check its ACLs

```
cd tp_acl/
touch test_inherit.txt
getfacl test_inherit.txt
```

Screenshot 6: Output of `getfacl test_inherit.txt`

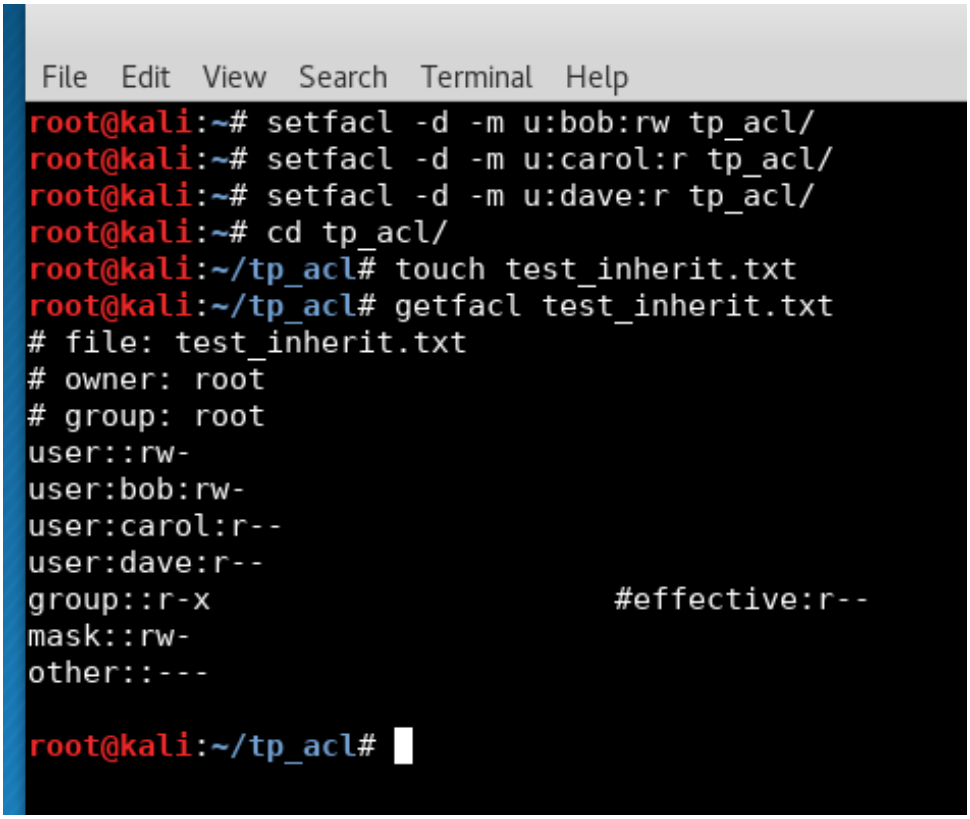


Figure 6: Default ACL inheritance verified on `test_inherit.txt`

Question: What happens if a user creates a new file in this directory?

Answer: When a user creates a new file inside `tp_acl/`, it automatically inherits the default ACLs defined for that directory. This means users `bob`, `carol`, and `dave` will have the same access rights (read/write or read-only) on any newly created file without needing to set ACLs manually.

Exercise 6 – Practical Case (Collaboration)

Context: A shared project must be hosted in `/home/[first_last]/project_acl/` with the following rules:

- `alice`: full rights (owner)
- `bob`: read + write
- `carol`: read only
- `dave`: read without execute
- New files created inside should automatically inherit these permissions

1. Create the folder and apply the ACLs

```
mkdir /home/Mostefaoui_Manel/project_acl/
setfacl -m u:alice:rwx /home/Mostefaoui_Manel/project_acl/
setfacl -m u:bob:rw /home/Mostefaoui_Manel/project_acl/
setfacl -m u:carol:r /home/Mostefaoui_Manel/project_acl/
setfacl -m u:dave:r /home/Mostefaoui_Manel/project_acl/
```

2. Configure default ACLs for inheritance

```
setfacl -d -m u:alice:rwx /home/Mostefaoui_Manel/project_acl/
setfacl -d -m u:bob:rw /home/Mostefaoui_Manel/project_acl/
setfacl -d -m u:carol:r /home/Mostefaoui_Manel/project_acl/
setfacl -d -m u:dave:r /home/Mostefaoui_Manel/project_acl/
```

3. Configure the mask to restrict execution to only alice

```
setfacl -m m:rw /home/Mostefaoui_Manel/project_acl/
```

4. Verify the ACLs

```
getfacl /home/Mostefaoui_Manel/project_acl/
```

Screenshot 7: Output of `getfacl /home/MostefaouiManel/project_acl/`

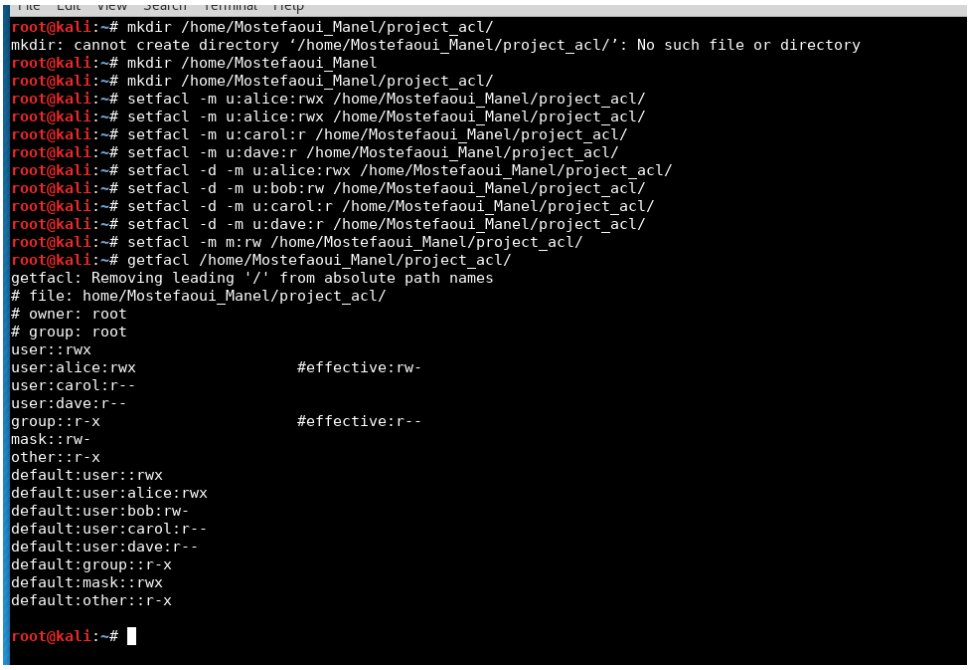


Figure 7: ACL configuration for collaborative project directory `project_acl`